«ΣΥΣΤΗΜΑΤΑ ΔΙΑΧΕΙΡΙΣΗΣ ΒΑΣΕΩΝ ΔΕΔΟΜΕΝΩΝ»

Εκπαιδευτής: Κωνσταντίνος Αναστασίου

Εξάμηνο: Δ

Βασικές εντολές SQL

- ✓ δημιουργία βάσης (CREATE DATABASE)
- ✓ επιλογή βάσης (USE)
- ✓ δημιουργία πίνακα (CREATE TABLE)
- ✓ εισαγωγή δεδομένων (INSERT)
- ✓ διαγραφή δεδομένων (DELETE)
- ✓ διαγραφή πίνακα (DROP TABLE)

Ύλη & Σημειώσεις: Κωνσταντίνος Αναστασίου

1. Βασικές έννοιες σύνταξης SQL (πριν τις εντολές)

1.1 Identifiers (ονόματα)

Identifiers είναι τα ονόματα που δίνουμε σε:

- βάσεις δεδομένων (databases)
- πίνακες (tables)
- στήλες (columns)

Παραδείγματα identifiers:

- school_db (βάση)
- students (πίνακας)
- first_name (στήλη)

Κανόνες καλής πρακτικής:

- μόνο λατινικούς χαρακτήρες: students όχι μαθητές
- χωρίς κενά: first name (λάθος) → first_name (σωστό)
- συνέπεια στο naming (snake_case)

1.2 Literals (τιμές)

Literals είναι οι πραγματικές τιμές που γράφουμε σε INSERT/DELETE:

- κείμενο: 'Maria'
- αριθμός: 20
- ημερομηνία: '2026-02-25'

Κανόνας:

- κείμενα και ημερομηνίες σε **μονά εισαγωγικά** '...'
- αριθμοί χωρίς εισαγωγικά

1.3 Σχόλια (comments)

Χρήσιμα για να εξηγείται ο κώδικας:

- Μονής γραμμής: -- σχόλιο
- Πολλαπλών γραμμών: /* σχόλιο */

---

2. Δημιουργία Βάσης Δεδομένων (CREATE DATABASE)

2.1 Σύνταξη

CREATE DATABASE όνομα_βάσης;

Παράδειγμα:

CREATE DATABASE school_db;

Τι κάνει:

- δημιουργεί έναν “χώρο” που θα περιέχει πίνακες (tables)

2.2 Αν η βάση υπάρχει ήδη

Για να μην εμφανιστεί σφάλμα:

CREATE DATABASE IF NOT EXISTS school_db;

---

3. Επιλογή Βάσης (USE)

Πριν δημιουργηθούν πίνακες, πρέπει να “ενεργοποιηθεί” η βάση:

USE school_db;

Αν δεν γίνει USE, οι εντολές CREATE TABLE μπορεί να αποτύχουν ή να δημιουργήσουν πίνακες σε άλλη βάση.

---

4. Δημιουργία Πίνακα (CREATE TABLE)

4.1 Τι είναι ο πίνακας

Πίνακας = δομημένη μορφή αποθήκευσης.

- στήλες: “τι πληροφορία κρατάμε”
- γραμμές: “μία εγγραφή”

4.2 Σύνταξη

```
CREATE TABLE όνομα_πίνακα (
  στήλη1 Τύπος Περιορισμοί,
  στήλη2 Τύπος Περιορισμοί,
```

```
  ...
);
```

Παράδειγμα πίνακα μαθητών:

```
CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  age INT,
  email VARCHAR(100) UNIQUE
);
```

---

5. Αναλυτική εξήγηση "όλων των parts" του CREATE TABLE

5.1 Στήλες (columns)

Κάθε γραμμή μέσα στις παρενθέσεις είναι μία στήλη.

Παράδειγμα:

first_name VARCHAR(50) NOT NULL

Σπάσιμο σε κομμάτια:

- first_name → όνομα στήλης (identifier)
- VARCHAR(50) → τύπος δεδομένων
- NOT NULL → περιορισμός (constraint)

5.2 Τύποι δεδομένων (data types) που θα χρησιμοποιούνται από την αρχή

Ακέραιοι

- INT
  Ακέραιος αριθμός (π.χ. 1, 20, 300)

Χρήση: ηλικία, id, ποσότητες.

Δεκαδικοί

- DECIMAL(p, s)
  p = συνολικά ψηφία
  s = πόσα από αυτά είναι δεκαδικά

Παράδειγμα:

- DECIMAL(8,2) επιτρέπει μέχρι 999999.99

Χρήση: τιμές, χρήματα.

Κείμενο (συνηθέστερο)

- VARCHAR(n)
  Κείμενο μεταβλητού μήκους μέχρι n χαρακτήρες.

Παράδειγμα:

- VARCHAR(50) για όνομα
- VARCHAR(100) για email
- TEXT
  Κείμενο μεγάλου μήκους (π.χ. περιγραφές)

Ημερομηνίες / Ώρες

- DATE (YYYY-MM-DD)
- DATETIME (YYYY-MM-DD HH:MM:SS)

Χρήση: ημερομηνία εγγραφής, ημερομηνία γέννησης, timestamp συναλλαγής.

Ναι/Όχι

- BOOLEAN (στη MariaDB πρακτικά αποθηκεύεται ως 0/1)

Χρήση: ενεργός χρήστης, ολοκληρωμένο, διαθέσιμο κ.λπ.

---

6. Περιορισμοί (Constraints) – τι σημαίνουν

6.1 PRIMARY KEY

id INT PRIMARY KEY

- Μοναδικό για κάθε γραμμή
- Δεν επιτρέπεται να επαναλαμβάνεται
- Χρησιμοποιείται για ταυτοποίηση εγγραφών

6.2 AUTO_INCREMENT

id INT AUTO_INCREMENT

- Αν δεν δώσουμε τιμή στο id, η βάση βάζει την επόμενη (1, 2, 3, …)
- Χρήσιμο για ids

Συνήθως μπαίνει μαζί με PRIMARY KEY.

6.3 NOT NULL

first_name VARCHAR(50) NOT NULL

- Απαγορεύει “κενή” τιμή (NULL)
- Δεν επιτρέπει να αφήσουμε τη στήλη άδεια στην εγγραφή

6.4 UNIQUE

email VARCHAR(100) UNIQUE

- Δεν επιτρέπει δύο ίδιες τιμές
- Χρήσιμο σε email, usernames, ΑΦΜ κ.λπ.

---

7. Εισαγωγή Δεδομένων (INSERT)

7.1 Σύνταξη

```
INSERT INTO table_name (col1, col2, col3)
VALUES (val1, val2, val3);
```

Παράδειγμα:

```
INSERT INTO students (first_name, last_name, age, email)
VALUES ('Maria', 'Papadopoulou', 20, 'maria@mail.com');
```

Επεξήγηση:

- students = πίνακας
- (first_name, last_name, age, email) = σε ποιες στήλες θα μπει τιμή
- VALUES (...) = οι τιμές με την ίδια σειρά

Σημαντικό:

- Αν μια στήλη είναι AUTO_INCREMENT, δεν χρειάζεται να την δηλώσουμε.

7.2 Πολλαπλές εγγραφές σε ένα INSERT

```
INSERT INTO students (first_name, last_name, age, email)
VALUES
('Giannis', 'Ioannou', 22, 'giannis@mail.com'),
('Eleni', 'Kosta', 19, 'eleni@mail.com');
```

---

8. Διαγραφή Δεδομένων (DELETE)

8.1 Σύνταξη

```
DELETE FROM table_name
WHERE condition;
```

ΠΡΟΣΟΧΗ:
Αν δεν υπάρχει WHERE, διαγράφονται όλες οι γραμμές.

8.2 Παράδειγμα: διαγραφή ενός μαθητή με id

```
DELETE FROM students
WHERE id = 2;
```

---

9. Τελεστές (Operators) για συνθήκες στο WHERE

Οι τελεστές χρησιμοποιούνται για να γράφουμε συνθήκες (conditions).

9.1 Τελεστές σύγκρισης

- = ίσο
- != ή <> διαφορετικό
- > μεγαλύτερο
- < μικρότερο
- >= μεγαλύτερο ή ίσο
- <= μικρότερο ή ίσο

Παραδείγματα:

```
DELETE FROM students WHERE age > 21;
DELETE FROM students WHERE last_name = 'Kosta';
```

9.2 Λογικοί τελεστές

- AND (και)
- OR (ή)
- NOT (όχι)

Παραδείγματα:

```
DELETE FROM students
WHERE age >= 20 AND last_name = 'Ioannou';
```

```
DELETE FROM students
WHERE NOT age = 20;
```

9.3 Τελεστής IN

Χρήσιμο όταν ψάχνουμε/επιλέγουμε πολλαπλές τιμές:

```
DELETE FROM students
WHERE id IN (1, 3, 5);
```

---

10. Διαγραφή Πίνακα (DROP TABLE)

10.1 Σύνταξη

```
DROP TABLE table_name;
```

Παράδειγμα:

```
DROP TABLE students;
```

Το DROP TABLE:

- διαγράφει οριστικά τον πίνακα
- χάνεται και η δομή και τα δεδομένα

Για αποφυγή σφάλματος αν δεν υπάρχει:

```
DROP TABLE IF EXISTS students;
```

---

11. Πλήρες παράδειγμα (εργαστηριακή ροή)

```
CREATE DATABASE IF NOT EXISTS school_db;
USE school_db;
```

```sql
DROP TABLE IF EXISTS students;

CREATE TABLE students (
  id INT PRIMARY KEY AUTO_INCREMENT,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  age INT,
  email VARCHAR(100) UNIQUE
);

INSERT INTO students (first_name, last_name, age, email)
VALUES
('Maria', 'Papadopoulou', 20, 'maria@mail.com'),
('Giannis', 'Ioannou', 22, 'giannis@mail.com'),
('Eleni', 'Kosta', 19, 'eleni@mail.com');

DELETE FROM students WHERE id = 2;
```